

XRPL Validator — Daily Engineering Report • 2026-09-11

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **13 commits** (xrpl-validator 13)
- **12 engine findings fixed** (F252–F265)
- 1 test / vector commit
- Amendment **fixCleanup3_3_0** was enabled at ledger 106,911,489 (2026-09-11 11:29 UTC)

Engine findings fixed

- **fix(xrpl-ledger): F252 — a strand that builds but flows nothing is tecPATH_PARTIAL without tfPartialPayment; only a strand that fails to build is tecPATH_DRY** (5ca15a7 , xrpl-validator) Five flow-time zero guards in payment.rs answered tecPATH_DRY whatever the flags said: the sender holds nothing of the SendMax asset, the destination can receive nothing (at or over its trust limit), nothing lands after the transfer rate. rippled 3.3.0 judges the flag first — StrandFlow.h:812-827 `if (actualOut != outReq) { if (!partialPayment) return tecPATH_PARTIAL; else if (actualOut == 0) return tecPATH_DRY; } ...`
- **fix(xrpl-ledger): F253 — the AccountTxnID stamp answers the field's presence before do_apply; the AccountSet that arms it lands the zero hash** (fd60afd , xrpl-validator) rippled writes the sender's AccountTxnID in Transactor::apply, AHEAD of doApply (Transactor.cpp:906, `if (sle->isFieldPresent(sfAccountTxnID)) sle->setFieldH256(sfAccountTxnID, ctx_.tx.getTransactionID())`). An AccountSet with SetFlag 5 then creates the field in doApply by `makeFieldPresent` (AccountSet.cpp:380-383) — the zero hash — so the arming transaction is never itself recorded; the next one is.
- **fix(xrpl-ledger): F254 — AMMDeposit's per-amount requireAuth is the Legacy check; no line to a permissive issuer is tecUNFUNDED_AMM, not tecNO_LINE** (f5cef49 , xrpl-validator) AMMDeposit::preclaim's `checkAmount` (AMMDeposit.cpp:321, rippled 3.3.0) calls `requireAuth(ctx.view, amount->asset(), accountID)` with the DEFAULT AuthType::Legacy (TokenHelpers.h:340). For an IOU that means (RippleStateHelpers.cpp requireAuth): a missing line is tecNO_LINE only when the issuer has `lsfRequireAuth`; a permissive issuer passes, and the `balance` lambda then finds `accountFunds = 0` and answers ...
- **fix(xrpl-ledger): F255 — an IOU hop whose source holds nothing, or whose destination sits at its limit, is tecPATH_DRY at strand build; only the XRP endpoint is a flow-time zero** (7ed1439 , xrpl-validator) Finding 252 turned five flow-time zero guards partial-aware on the strength of StrandFlow.h's ordering. Three of them are not flow-time. A DirectIPaymentStep's `check` (DirectStep.cpp:450-460, rippled 3.3.0) judges the hop at strand BUILD: `owed = creditBalance(dst, src); if (owed <= 0 && -owed >= creditLimit(dst, src)) return tecPATH_DRY` — the step's source holds nothing of the counterparty's IOU and the ...

- **fix(xrpl-ledger): F256 — the last LP redeeming its whole position by tflPToken tears the AMM down** (7263d48 , xrpl-validator) A tflPToken withdrawal whose LPTokenIn equals the pool's LPTokenBalance is rippled's full withdrawal (AMMWithdraw.cpp:852: both balances paid out whole), and `deleteAMMAccountIfEmpty` (:800-820) then deletes the pool's trust lines, the AMM object, its owner directory and the pool account once the new LPTokenBalance is zero. Our `tfWithdrawAll` arm already did that (#106430239); the tflPToken path shares the general ...
- **fix(xrpl-ledger): F258 — IOU value strings spell like the decoder (STAmount::getText), and threading compares amounts as numbers** (5bfb88c , xrpl-validator) `me_to_value_string` printed every non-negative exponent as decimal with at most 40 zeros, so any amount above 1e56 was silently rewritten as 1e-40 of itself the moment the engine re-serialised the object holding it. STAmount's exponent runs to +80 — a 16-digit mantissa makes 9999999999999999e80 — and mainnet's gateway "bridge" offers are sized exactly there.
- **fix(xrpl-ledger): F257 — a bridged leg's single-path upper bound is the anchored pool offer's quality, not the book tip's** (5cb9280 , xrpl-validator) rippled admits a strand by `qualityUpperBound`, and for a BookStep whose pool is live that is `tip() = getOffer(lobQuality)`: the offer `changeSpotPriceQuality` sizes AGAINST the book tip, whose average quality sits between the pool's spot and the tip (AMMLiquidity.cpp:184-222). Only the `fixAMMv1_1` `qualityThreshold` override — or an empty book — hands it the spot. Finding 99 gave the direct walk that rule ...
- **feat(xrpl-ledger): F259 — fixCleanup3_3_0 is live; read amendment state from the ledger and apply its AMM deposit/withdraw rules** (703df39 , xrpl-validator) `fixCleanup3_3_0` was enabled on mainnet at flag ledger 106912768 (2026-09-11) with `deploy117` already running. The engine had no notion of amendment state — it hard-coded the pre-amendment rules it had specimens for — so it now reads the ledger's Amendments singleton (keylet 7DB0788C..., `sfAmendments`), rippled's `Rules::enabled`. A state without the singleton (a bundle drill's seated pre-images, a synthetic test ledger) ...
- **fix(xrpl-ledger): F260 — a taker crossing its own offer moves nothing on the line; only the flow's remainingIn falls** (a967bac , xrpl-validator) rippled's `accountSendIOU` returns before touching anything when the sender is the receiver (TokenHelpers.cpp:1451), transfer fee included. A payment that crosses the sender's OWN offer therefore leaves the sender's line exactly where it was, while the strand's `actualIn` still counts against `remainingIn` (StrandFlow.h). The direct walk credited the maker per fill and debited the taker once at pass end — the same line, ...
- **fix(xrpl-ledger): F262 — a trust limit filed at STAmount's ceiling is not a 0.034 cap; limit – held is an IOUAmount subtraction** (453e1ca , xrpl-validator) `dest_receivable` (the strand's last-step ceiling, DirectStep.cpp:476-488) and `max_src_to_dst` computed "limit – held" with `me_sub`, which rescales the limit to the holding's exponent through `me_rescale` — capped at 38 orders and SATURATING. rapido files its limits at STAmount's ceiling, 9999999999999999e79; against a 1e-25 holding that is 104 orders, the limit became `u128::MAX` and the room ...
- **fix(xrpl-ledger): F265 — redeeming an IOU to its issuer reads the holding through IgnoreFreeze; a frozen holder can still pay the gateway back** (55bc125 , xrpl-validator) Paying the issuer is a one-step strand. rippled sizes a DirectStepI from `accountHolds(..., FreezeHandling::IgnoreFreeze)` (DirectStep.cpp `maxPaymentFlow`) and enforces freezes at strand build with `checkFreeze`, which a one-step strand skips — "pure issue/redeem can't be frozen". Our `available` reads holders through `fhZERO_IF_FROZEN` — right for offer funding and for the two-step strand, whose `hop_frozen_parts` ...
- **fix(xrpl-ledger): F264 — DepositPreauth(Authorize) counts against the creator's owner reserve** (6d2a42e , xrpl-validator) rippled's `DepositPreauth::doApply` runs `checkReserve(sleOwner, preFeeBalance_`,

{ownerCountDelta = 1}) before creating the entry (DepositPreauth.cpp:168-173): the balance BEFORE the fee must cover the reserve at OwnerCount + 1, so the fee may dip into the reserve but the new object may not. We created the entry unconditionally.

Tests and campaigns

- **test(xrpl-node): repair the F260/F262 merge in payment_flow_vector — the self-cross test lost its closing brace** (59cfa22 , xrpl-validator)

Amendment watch

Read from the XRP Ledger: the Amendments object as of the last ledger of the day, and the day's EnableAmendment pseudo-transactions.

Voting events:

- fixCleanup3_3_0 was enabled — ledger 106,911,489, 2026-09-11 11:29 UTC

No amendment held a majority.

Amendments enabled on mainnet: 94.

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Per-Signer-List Spend Policies** (accounts) Add a new owner-reserved SignerListPolicy SLE (keylet::signerListPolicy(account, signerListID)) settable via an optional payload on SignerListSet, c
- **Atomic NAV-Priced Fund Redemption** (rwa) Add a FundNAV ledger object (owner-reserved by the fund issuer, keylet::fundNAV(issuanceID)) holding sfNAVPerShare, sfNAVSequence, and sfSettlementA
- **Bridge-Attested Wrapped Assets & Messages** (interop) Extend the existing XChainClaimAttestation template with an optional bounded sfBridgeMessage (≤1KB) and sfMPTIssuanceID field, so a witness-attested X

Appendix — commits

Time	Repo	Commit	Subject
01:50	xrpl-validator	5ca15a7	fix(xrpl-ledger): F252 — a strand that builds but flows nothing is tecPATH_PARTIAL without tfPartialPayment; only a strand that fails to build is tecPATH_DRY
01:58	xrpl-validator	fd60afd	fix(xrpl-ledger): F253 — the AccountTxnID stamp answers the field's presence before do_apply; the AccountSet that arms it lands the zero hash
04:18	xrpl-validator	f5cef49	fix(xrpl-ledger): F254 — AMMDeposit's per-amount requireAuth is the Legacy check; no line to a permissive issuer is tecUNFUNDED_AMM, not tecNO_LINE

Time	Repo	Commit	Subject
04:28	xrpl-validator	7ed1439	fix(xrpl-ledger): F255 — an IOU hop whose source holds nothing, or whose destination sits at its limit, is tecPATH_DRY at strand build; only the XRP endpoint is a flow-time zero
05:26	xrpl-validator	7263d48	fix(xrpl-ledger): F256 — the last LP redeeming its whole position by tfLPToken tears the AMM down
06:49	xrpl-validator	5bfb88c	fix(xrpl-ledger): F258 — IOU value strings spell like the decoder (STAmount::getText), and threading compares amounts as numbers
08:01	xrpl-validator	5cb9280	fix(xrpl-ledger): F257 — a bridged leg's single-path upper bound is the anchored pool offer's quality, not the book tip's
09:48	xrpl-validator	703df39	feat(xrpl-ledger): F259 — fixCleanup3_3_0 is live; read amendment state from the ledger and apply its AMM deposit/withdraw rules
10:19	xrpl-validator	a967bac	fix(xrpl-ledger): F260 — a taker crossing its own offer moves nothing on the line; only the flow's remainingIn falls
12:19	xrpl-validator	453e1ca	fix(xrpl-ledger): F262 — a trust limit filed at STAmount's ceiling is not a 0.034 cap; limit – held is an IOUAmount subtraction
12:28	xrpl-validator	59cfa22	test(xrpl-node): repair the F260/F262 merge in payment_flow_vector — the self-cross test lost its closing brace
21:48	xrpl-validator	55bc125	fix(xrpl-ledger): F265 — redeeming an IOU to its issuer reads the holding through IgnoreFreeze; a frozen holder can still pay the gateway back
21:48	xrpl-validator	6d2a42e	fix(xrpl-ledger): F264 — DepositPreauth(Authorize) counts against the creator's owner reserve